AI PRIVACY FIREWALL – MIDNIGHT ATTESTATION

Privacy-Aware Decision Logging and Blockchain-Based Audit Integrity

PROJECT REPORT

Submitted in partial fulfilment of the requirements for
[SUBJECT / MINI-PROJECT NAME]
in the degree of Bachelor of Engineering in Computer Engineering

Submitted by

Arya Patel

Roll No.: **[ROLL NUMBER]**

Under the guidance of

[FACULTY GUIDE NAME]

Department of Computer Engineering
Shree L. R. Tiwari College of Engineering

[MONTH YEAR OF SUBMISSION]

1. PROBLEM STATEMENT

1.1 Background

AI agents are increasingly connected to business data. A common safeguard is an instruction in the model's system prompt asking it not to reveal sensitive values. That is guidance, not a control: an injected prompt or a misbehaving model can ignore it, and afterwards there is often no reliable record of what data the agent was allowed to see.

1.2 Problem Addressed

This project addresses two linked problems:

- **Privacy:** an agent should receive only the fields it needs, in the least revealing form (raw value, aggregate, or nothing), and its responses should be checked for leaked secrets or personal data before they reach the user.
- **Audit integrity:** every access decision should be recorded so that later tampering (editing, reordering or deleting entries) is detectable, and the decisions should be attestable to a third party without publishing the underlying data.

1.3 Objectives

- Enforce field-level, fail-closed access policies outside the model, from a YAML policy file.
- Minimize data: release raw values, `{count, sum, avg}` aggregates with a k-anonymity floor, or nothing.
- Inspect model output for canary tokens and unauthorized personal data, blocking violations.
- Record every decision in a SHA-256 hash chain whose tampering is detectable.
- Reduce each decision to a 32-byte hash suitable for a Midnight Compact contract, keeping private details local.
- Compile the Compact contracts with the real toolchain and verify their logic in a local simulation.

2. PROJECT DESCRIPTION

The project has three parts, shown in Figure 2.1. Part A is the Python privacy firewall in `privacy_firewall/`. Part B is a set of standalone audit utilities in `firee/midnight-side/audit/` that hash-chain and digitally sign an audit log. Part C is two Midnight Compact contracts, compiled with the Compact toolchain and executed locally by a simulation script.

2.1 Workflow

1. An AI agent sends a structured request (agent, resource, operation, fields, query). It never queries the data store itself.
2. `PrivacyFirewallMiddleware` asks the `PolicyEngine` for a decision on each field and passes the rows of `MockEmployeeDB` through the `DataMinimizer`.
3. The minimized payload goes to the model. In this project the model is `FakeLLM`, a deterministic stand-in defined in `demo.py`, so behaviour is reproducible and offline.
4. The `OutputFirewall` scans the model's response for canary tokens and unauthorized personal data and returns ALLOW or BLOCK.

5. Every field decision is appended to the `AuditLog` hash chain.
6. The `MidnightBridge` turns each decision into a `Bytes32` hash and an HMAC-signed batch; `export_attestation_batch.py` writes that batch to a file.
7. `simulate_contract.ts` loads the compiled contracts and records the batch's hashes with `recordAttestation`, then checks them with `isAttested`. It also commits the audit tools' root hash to `commit.compact`.

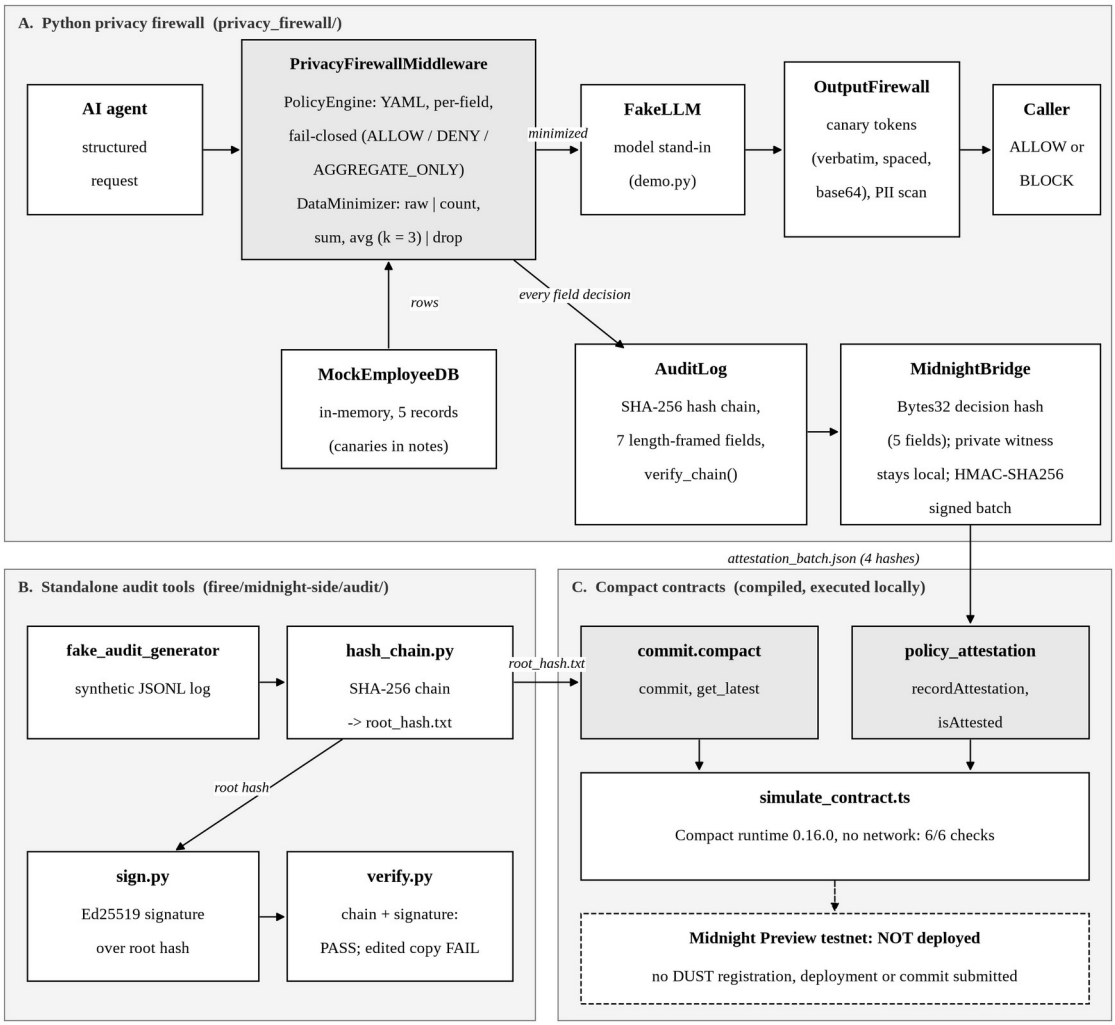


Figure 2.1: *AI Privacy Firewall system architecture and data flow*

2.2 Local Simulation versus Blockchain Deployment

All contract results in this report come from executing the compiled contracts locally with the Compact runtime. No transaction was submitted to any Midnight network. A wallet for the Midnight Preview test network was created, funded from the official faucet and synchronized, and a DUST registration transaction was built in a dry run, but DUST registration, contract deployment and the on-chain audit commitment were not carried out. There is therefore no deployed contract address.

3. TECHNOLOGY USED

Table 3.1: *Technologies used in the project*

TECHNOLOGY	DESCRIPTION
Python 3	Language of the firewall, audit chain and bridge. Uses the standard library for hashing (`hashlib`), HMAC (`hmac`) and random tokens (`secrets`).
PyYAML	Reads the per-agent access policy (`policies/analytics_agent.yaml`).
pytest	Runs the 234 automated tests in `privacy_firewall/tests/`.
cryptography (Python)	Provides Ed25519 signing and verification for the standalone audit tools (`audit/sign.py`, `audit/verify.py`).
Midnight Compact	Smart-contract language for Midnight. The toolchain (version 0.31.1) compiles each circuit into JavaScript plus zero-knowledge prover and verifier keys.
Compact runtime and compact-js	JavaScript libraries (`compact-runtime` 0.16.0, `compact-js` 2.5.1) that execute compiled contracts; used by the offline simulation.
TypeScript, Node.js 22, tsx	Language and runtime of the Midnight-side scripts (simulation, wallet, deployment pipeline); `tsc` performs the type check.
Midnight.js 4.1.1 and Wallet SDK	Midnight's client libraries for wallets, transactions and contract calls. Used for wallet synchronization and the registration dry run; no transaction was submitted.
Midnight proof server 8.1.3	Local service, run in a Podman container, that generates zero-knowledge proofs for live transactions. It was started and health-checked; it has not proved any submitted transaction.
GitHub Actions	CI workflow (`.github/workflows/ci.yml`) that runs the tests and demo on Python 3.10–3.12. It was not run as part of this report's verification.

3.1 Environment and Limitations

All verification was performed on a single Fedora Linux 44 machine with Python 3.14.8, Node.js 22.23.4 and the Compact toolchain 0.31.1, versions chosen to match Midnight's published compatibility matrix. There is no server, database, web interface or deployed application: the firewall is a Python library with a command-line demo, its data source is an in-memory mock of five employee records, and the model is a deterministic stub rather than a real language model.

4. ALGORITHM AND TECHNICAL METHODS

4.1 Policy Decision (Fail-Closed)

For every requested field the `PolicyEngine` returns one of three decisions from the agent's YAML policy:

1. If the agent has no policy, return DENY ("failing closed").
2. If the field is not listed in the agent's policy, return DENY.
3. Otherwise return the listed decision: ALLOW, AGGREGATE_ONLY or DENY.

4.2 Data Minimization

ALLOW fields pass through unchanged; DENY fields are dropped. AGGREGATE_ONLY fields are reduced to aggregates. Minimum and maximum are deliberately not returned because they would reveal one person's exact value, and groups smaller than k = 3 are suppressed to a count only (`data_minimizer.py`, `_aggregate`).

4.3 Output Inspection

The `OutputFirewall` blocks a response (decision BLOCK, output withheld) if it contains a canary token, a secret planted in the data that no agent is authorized to see, or an unmasked personal-data entity outside the agent's allowed fields. Canaries are generated with `secrets.token_hex` and are detected verbatim, with spaces or newlines inserted, and base64-encoded. The `PIIDetector` finds emails, phone numbers, US social security numbers, credit-card numbers (13–19 digits with a Luhn check), person names (a capitalized-name heuristic) and API keys or secrets (labels and high-entropy tokens).

4.4 Firewall Audit Hash Chain

Each decision becomes an `AuditEntry` whose hash covers seven fields and the previous entry's hash, starting from a genesis value of 64 zeros. Each field is prefixed with its length, so a delimiter character inside a field cannot make two different entries produce the same hash input:

```
parts = [entry_id, agent, field_name, decision.value, reason, timestamp, prev_hash]
return "|".join(f"{len(p)}:{p}" for p in parts)
```

`verify_chain()` recomputes every hash in order; any edited, reordered or deleted entry breaks the chain and verification returns false. This chain lives in memory and is not digitally signed.

4.5 Decision Hash and Attestation Batch

For the contract, the `MidnightBridge` hashes five fields of each decision (agent, field, decision, reason, timestamp) with the same length framing, giving a 32-byte SHA-256 value. Agent, field names and reason stay in the private part of a local witness; only the hash is meant for the chain. The batch of hashes is signed with HMAC-SHA256 using a development key by default. HMAC proves the batch was produced by a holder of that key and has not changed; it is not a zero-knowledge proof.

4.6 Standalone Audit Signing and Tamper Detection

The utilities in `firee/midnight-side/audit/` operate on a synthetic JSONL audit log, separate from the firewall's in-memory chain:

1. `hash_chain.py`: for each record, h = SHA-256(canonical JSON of record + previous h), starting from 64 zeros; the final h is the root hash.
2. `sign.py`: sign the root hash bytes with an Ed25519 private key.
3. `verify.py`: rebuild the chain from the log, compare it with the expected root hash, and check the Ed25519 signature. Both checks must pass; otherwise the result is FAIL.

4.7 Contract Logic

`policy_attestation.compact` stores attested decision hashes in a public map; `disclose()` marks only the hash as public. `commit.compact` stores the latest root hash and a commit counter.

```
export ledger attested: Map<Bytes<32>, Boolean>;

export circuit recordAttestation(decisionHash: Bytes<32>): [] {
  attested.insert(disclose(decisionHash), true);
}

// Read-only check that a specific decision hash was, in fact, recorded.
export circuit isAttested(decisionHash: Bytes<32>): Boolean {
  return attested.member(disclose(decisionHash));
}
```

5. IMPLEMENTATION

5.1 Python Modules

Table 5.1: *Main Python files (privacy_firewall/)*

FILE	ROLE
`policy_engine.py`	Loads the YAML policy; fail-closed per-field decisions
`data_minimizer.py`	Raw, aggregate (count, sum, avg; k = 3) or dropped values
`pii_detector.py`	Regex, Luhn and entropy-based detection; three masking strategies
`canary.py`	Generates and checks canary tokens
`output_firewall.py`	Canary and personal-data scan of model output; ALLOW or BLOCK
`agent.py`	`MockEmployeeDB`, `AIAgent`, `PrivacyFirewallMiddleware`
`audit.py`	SHA-256 hash-chained audit log and `verify_chain()`
`midnight_bridge.py`	Decision hashes, private witness, HMAC-signed batch, contract interface check
`demo.py`	Five-scenario command-line demonstration with `FakeLLM`
`export_attestation_batch.py`	Runs firewall requests and exports the signed batch

5.2 Contracts and Midnight-Side Scripts

- **`privacy_firewall/midnight/policy_attestation.compact`:** circuits `recordAttestation` and `isAttested`.
- **`firee/midnight-side/contract/commit.compact`:** circuits `commit` and `get_latest`.
- **`firee/midnight-side/scripts/simulate_contract.ts`:** executes both compiled contracts locally.
- **`firee/midnight-side/scripts/`:** also contains the wallet, synchronization, DUST-registration, fee estimation, deployment and commit scripts prepared for an optional live deployment.

5.3 Integration Points

The bridge parses `policy_attestation.compact` at start-up and refuses to run if the contract lacks the `recordAttestation` circuit, so the Python side and the contract interface cannot silently diverge. The exported batch file connects Part A to Part C, and `root_hash.txt`

connects Part B to Part C (Figure 2.1). Because the batch contains the private witness, it is excluded from version control.

5.4 Running the Project

```
cd privacy_firewall
python -m pytest -q                       # 234 tests
python demo.py --no-pause --no-color      # 5 scenarios
PYTHONPATH=. python export_attestation_batch.py
../firee/midnight-side/build/attestation_batch.json

cd ../firee/midnight-side
npm run compile && npm run compile:policy # both contracts
npm run typecheck
npm run simulate                          # 6 checks, no network
```

Listing 5.1: *Verification commands (full setup in SUBMISSION.md)*

6. RESULTS AND DISCUSSION

All results below were obtained locally and offline on 29 October 2026 and re-run that day during preparation of this report with identical outcomes. Verbatim terminal output is stored in `docs/evidence/`.

Table 6.1: *Verified results*

CHECK	RESULT	SCOPE
Python test suite	234 passed, 0 failed	Local
Command-line demo	5 of 5 scenarios PASS	Local
Compile `commit.compact`	Compiled; 2 circuits with prover/verifier keys	Local
Compile `policy_attestation.compact`	Compiled; 2 circuits with prover/verifier keys	Local
TypeScript type check	No errors	Local
Contract simulation	6 of 6 checks pass (4 firewall decision hashes)	Local, no network
Audit verification, original log	Chain PASS, signature PASS	Local
Audit verification, tampered copy	Chain FAIL, signature FAIL (expected)	Local
Midnight Preview deployment	Not performed	—

6.1 Discussion

- **234 tests:** the largest suite (65 tests) targets prompt injection, privilege escalation and canary exfiltration and asserts that no path leaks a canary or raw personal data (Figure 6.1). Passing tests show the implemented rules behave as specified; they do not prove security against every attack.
- **5 demo scenarios:** minimized aggregate access, a policy violation, a prompt-injection attempt blocked inbound and outbound, a canary leak blocked by the output firewall, and an audit export with a verified hash chain.
- **Two compiled contracts:** compilation with the real toolchain confirms the contracts are valid Compact and produces the zero-knowledge keys a deployment would need. During verification the policy contract was found not to compile and was corrected (Section 7.3).

- **6/6 simulation checks:** the contract logic stores and returns exactly the values given to it, including four hashes produced by the real firewall. This verifies the logic and the Python-to-contract data format, not network behaviour.
- **Audit verification:** PASS on the original log and FAIL on a copy with one changed field show that the hash chain and Ed25519 signature detect modification.

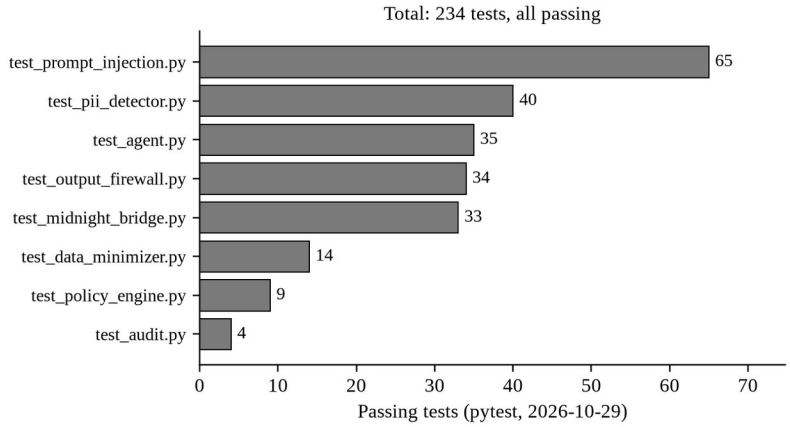


Figure 6.1: *Distribution of the 234 passing tests across the eight test suites*

7. TESTING AND VALIDATION

Table 7.1: *Checks, commands and results*

CHECK	PURPOSE	COMMAND	RESULT
Unit and security tests	Policy, minimizer, detector, firewall, audit, bridge and injection behaviour	`python -m pytest -q`	234 passed
Demo	End-to-end scenarios	`python demo.py --no-pause --no-color`	5/5 PASS
Contract compile	Valid Compact; keys generated	`npm run compile`, `npm run compile:policy`	Pass
Type check	TypeScript scripts are type-correct	`npm run typecheck`	Pass
Batch export	Real decision hashes for the contract	`export_attestation_batch.py`	4 hashes
Simulation	Contract logic with real hashes	`npm run simulate`	6/6 pass
Audit verify	Integrity of log and signature	`verify.py audit_log.jsonl <root> <sig>`	PASS
Tamper test	Detection of a modified log	`verify.py <edited copy> <root> <sig>`	FAIL (expected)

7.1 Audit Verification and Tamper Test

The root hash of the synthetic audit log was signed with Ed25519 and verified (exit code 0). One field of one record was then changed in a copy of the log (policy P-001 to P-999) and verified against the same root hash and signature. Verification failed with exit code 1, the expected result:

```
--- original log
Chain integrity:    PASS
Signature check:    PASS
OVERALL: PASS
--- copy with one field changed
Chain integrity:    FAIL
Signature check:    FAIL
OVERALL: FAIL - tamper detected or invalid signature
```

Listing 7.1: *Excerpt of docs/evidence/07_audit_sign_verify.txt*

7.2 Firewall Hashes through the Policy-Attestation Contract

`export_attestation_batch.py` ran two agent requests through the firewall, producing four decision hashes. The simulation recorded each with `recordAttestation`, confirmed `isAttested` is true for all four and false for a hash that was never recorded, and checked that the ledger holds exactly four entries. This was a local execution of the compiled contract, not an on-chain transaction.

```
commit.compact
  PASS  fresh contract: commitCount = 0
  PASS  get_latest() == audit/root_hash.txt (f9c0d1c44cd36650...)
  PASS  commitCount = 1 after commit
policy_attestation.compact
  PASS  isAttested() true for all 4 firewall decision hashes
  PASS  ledger holds exactly those hashes
  PASS  isAttested() false for an unrecorded hash
Simulation: all checks passed
```

Listing 7.2: *Excerpt of docs/evidence/06_simulate_contracts.txt*

7.3 Defects Found During Validation

Compiling `policy_attestation.compact` with the real toolchain failed: the standard library was not imported and the key type `Opaque<"Bytes32">` is not supported. It was corrected to `Bytes<32>`, the matching parser in `midnight_bridge.py` was updated, and all 234 tests still pass. The Midnight runtime dependency was also realigned to Midnight's compatibility matrix (`compact-runtime` 0.16.0).

Screenshots are not included; the evidence is captured terminal output. If screenshots are required, they can be taken of the commands in Listing 5.1.

8. CONCLUSION

The project shows that privacy rules for an AI agent can be enforced outside the model: the agent receives only what its policy allows, in the least revealing form, and responses that leak canaries or unauthorized personal data are blocked. Every decision is recorded in a tamper-evident hash chain and reduced to a 32-byte hash that a Midnight Compact contract can attest without revealing the underlying data.

These properties are verified by 234 automated tests, five demonstration scenarios, two contracts compiled with the real Compact toolchain, a local contract simulation using real firewall hashes, and an audit verification that accepts an untouched log and rejects a modified one. The guarantees are those of the implemented checks on a deterministic test setup; they are not a proof of security

against all attacks, and the blockchain step has been demonstrated locally rather than on a live network.

9. LIMITATIONS AND FUTURE SCOPE

9.1 Limitations

- Live Midnight Preview deployment, DUST registration and the on-chain audit commitment have not been completed; contract behaviour is shown by local simulation only.
- The attestation batch is signed with HMAC-SHA256 and a development key by default; it is not a zero-knowledge proof, and key management is not production-grade.
- The firewall's in-memory audit chain is not signed; Ed25519 signing is applied by the separate audit utilities to a synthetic log.
- The model is a deterministic stub and the data source is a five-record in-memory mock.
- Personal-data detection is rule-based (regular expressions, Luhn check, entropy) with limited recall.

9.2 Future Scope

- Optional testnet deployment: register the funded wallet for DUST, deploy both contracts and call `recordAttestation` and `commit` from the batch exporter.
- Sign the firewall's own audit chain and manage signing keys securely instead of using development keys.
- Replace the HMAC batch signature with a zero-knowledge proof.
- Broaden security testing, for example against real language models and larger, more varied datasets.
- Optional higher-recall personal-data detection and streaming output inspection.

10. REFERENCES

[1] Project repository documentation: `README.md`, `SUBMISSION.md`, `firee/midnight-side/README.md`, and verification output in `docs/evidence/`.

[2] Python Software Foundation, "hashlib — Secure hashes and message digests," https://docs.python.org/3/library/hashlib.html

[3] Python Software Foundation, "hmac — Keyed-Hashing for Message Authentication," https://docs.python.org/3/library/hmac.html

[4] Python Software Foundation, "secrets — Generate secure random numbers," https://docs.python.org/3/library/secrets.html

[5] pytest documentation, https://docs.pytest.org/

[6] PyYAML documentation, https://pyyaml.org/wiki/PyYAMLDocumentation

[7] Python Cryptographic Authority, "Ed25519 signing," https://cryptography.io/en/latest/hazmat/primitives/asymmetric/ed25519/

[8] Midnight Network documentation, https://docs.midnight.network/

[9] Midnight Network, "Release compatibility matrix," https://docs.midnight.network/relnotes/support-matrix

[10] Midnight Network, Compact toolchain releases, https://github.com/midnightntwrk/compact

[11] NIST, FIPS 180-4, "Secure Hash Standard (SHS)," https://csrc.nist.gov/pubs/fips/180-4/upd1/final

[12] H. Krawczyk, M. Bellare and R. Canetti, RFC 2104, "HMAC: Keyed-Hashing for Message Authentication," https://www.rfc-editor.org/rfc/rfc2104

[13] S. Josefsson and I. Liusvaara, RFC 8032, "Edwards-Curve Digital Signature Algorithm (EdDSA)," https://www.rfc-editor.org/rfc/rfc8032